

LINUX FUNDAMENTALS HANDBOOK

for AWS re/Start

15 Comprehensive Chapters

Commands • Labs • Interview Prep • AWS EC2 Scenarios

About This Handbook

This handbook is designed for students enrolled in the AWS re/Start programme and anyone working toward a career in cloud computing, Linux administration, or DevOps engineering.

Each chapter follows a consistent structure:

- Learning Objectives — what you will be able to do after the chapter
- Command Reference — practical tables of real commands with explanations
- Real AWS EC2 Scenarios — hands-on workflows used in production
- Interview Questions — the exact questions asked in technical interviews
- Chapter Summary — quick-revision checklist of key takeaways

How to Use This Book

Read each chapter sequentially - later chapters build on earlier ones. Practice every command in a real Linux environment (an AWS Free Tier EC2 instance is ideal). Revisit the Interview Questions section before job applications.

Mohammed Ammar

CHAPTER 1

Introduction to Linux

Learning Objectives

After completing this chapter, you will be able to:

- Understand what Linux is and its role in modern computing
- Explain the role of the Linux kernel
- Understand distributions and their purpose
- Navigate the Linux file system structure
- Understand the shell and command-line interface
- Identify common Linux distributions used in cloud environments
- Explain why Linux is widely used in AWS

1.1 What is Linux?

Linux is an open-source, Unix-like operating system built around the Linux kernel. It manages computer hardware and software resources while providing a secure and efficient platform for running applications.

An operating system acts as an interface between the user, application software, and computer hardware. It controls memory, processes, storage devices, filesystems, networking, and security, allowing applications to run efficiently.

Layer	Description
User	Person interacting with the system
Applications	Programs and software (Firefox, VS Code, MySQL)
Shell	Command interpreter that translates user commands
Kernel	Core of Linux — manages all hardware resources
Hardware	Physical components: CPU, RAM, Disk, Network

1.1.1 History of Linux

Linux was created by Linus Torvalds in 1991 while he was a computer science student at the University of Helsinki, Finland. Initially, Linux was developed as a personal project inspired by the UNIX operating system. Linus released the source code to the public under the GNU General Public License (GPL), allowing developers worldwide to contribute to its development. Today, Linux powers millions of servers, cloud platforms, smartphones, embedded systems, supercomputers, and Internet of Things (IoT) devices. Major cloud providers such as Amazon Web Services (AWS), Microsoft Azure, and Google Cloud rely heavily on Linux to deliver scalable and reliable cloud services.

1.1.2 GNU and Linux

Many beginners think Linux itself is the complete operating system. Technically, Linux is only the kernel.

The complete operating system combines the Linux kernel with the GNU utilities and libraries. GNU provides essential commands and tools such as Bash, ls, cp, mv, cat, grep, and many others.

For this reason, many people refer to the operating system as GNU/Linux.

Examples of GNU/Linux distributions include Ubuntu, Debian, Fedora, Red Hat Enterprise Linux (RHEL), Amazon Linux, Rocky Linux, and AlmaLinux.

1.2 Why Linux is Popular

Feature	Description
Free & Open Source	Anyone can view, modify, and distribute the source code
Secure	Strong security controls, permission management, and auditing
Stable	Servers run for months or years without rebooting
Scalable	Runs on laptops, data centres, supercomputers, and cloud platforms
Cloud Friendly	Most AWS EC2 instances use Linux operating systems

1.3 Real-World Examples

Company	Linux Usage
Google	Extensive use across all infrastructure and services
Amazon AWS	Core platform — majority of services run on Linux
Netflix	Streams content globally using Linux-based infrastructure
Facebook / Meta	Linux servers power their data centres worldwide

1.4 Linux Architecture

Linux consists of five key layers that work together:

```
User
|
Applications (Firefox | VS Code | MySQL)
|
Shell (Bash | Zsh, etc.)
|
Kernel (Process Management | Memory Management | Device Drivers | File System | Networking)
|
Hardware (PU | RAM | Disk | NIC)
```

1.4.1 Responsibilities of the Linux Kernel

The Linux kernel is the core component of the operating system and performs several important functions.

- Process Management – Creates, schedules, and terminates processes.
- Memory Management – Allocates and releases memory for running applications.
- Device Management – Communicates with hardware devices through device drivers.
- File System Management – Reads and writes data to storage devices.
- Network Management – Handles network communication using protocols such as TCP/IP.
- Security – Enforces user permissions and access control.

1.5 Common Linux Distributions

Distribution	Package Manager	Primary Use
Amazon Linux 2023	dnf	Recommended AWS EC2 distribution
Ubuntu	apt	Cloud, development, and general-purpose servers
Red Hat Enterprise Linux (RHEL)	dnf	Enterprise production environments
CentOS Stream	dnf	Enterprise testing and staging
Debian	apt	Stable, long-term server deployments
Fedora	dnf	Cutting-edge development
Kali Linux	apt	Security testing and penetration testing

Choosing the Right Distribution

Different Linux distributions are designed for different purposes.

- Amazon Linux is optimized for AWS cloud workloads.
- Ubuntu is beginner-friendly and widely used for cloud servers and development.
- Red Hat Enterprise Linux (RHEL) is commonly used in enterprise production environments with commercial support.
- Debian is known for its stability and reliability.
- Fedora provides the latest software and technologies for developers.
- Kali Linux includes hundreds of security testing tools for penetration testing and ethical hacking.

1.6 Linux File System Overview

In Linux, everything is a file — documents, directories, devices, and processes. The file system starts at the root directory /

Directory	Purpose	Example
/	Root — top of the entire file system	cd /
/home	User home directories	/home/ec2-user
/root	Home directory of the root (superuser)	/root
/etc	System-wide configuration files	/etc/passwd, /etc/ssh/sshd_config
/var	Variable data (logs, spools)	/var/log, /var/spool
/tmp	Temporary files (cleared on reboot)	/tmp
/usr	Installed applications and libraries	/usr/bin, /usr/lib
/bin	Essential commands available to all users	ls, cp, mv, cat

1.7 AWS EC2 Quick Start

Connect to a Linux EC2 instance via SSH:

```
# Connect using your key pair
ssh -i mykey.pem ec2-user@54.x.x.x

# Verify current user
whoami
# Output: ec2-user

# Show current directory
pwd
# Output: /home/ec2-user
```

Interview Questions

- Q What is Linux?
- Q What is the Linux kernel?
- Q What is a shell?
- Q What is a Linux distribution?
- Q What is the root directory (/)?
- Q Why is Linux popular in cloud computing?
- Q What is the difference between Linux and Windows?

Chapter Summary

In this chapter you learned:

- ✓ Linux is an open-source operating system widely used in cloud computing
- ✓ The kernel is the core of Linux, managing all hardware resources
- ✓ The shell acts as a command interpreter between user and kernel
- ✓ Linux distributions package the kernel with utilities and tools
- ✓ Everything in Linux is represented as a file
- ✓ The file system starts at the root directory (/)
- ✓ Amazon Linux 2023 and Ubuntu are the most common AWS distributions

CHAPTER 2

Linux Command Line Fundamentals

Learning Objectives

After completing this chapter, you will be able to:

- Navigate the Linux command line with confidence
- Understand Linux command syntax (command [options] [arguments])
- Use help and documentation commands
- Display system information
- Navigate directories and find command locations
- Use command history effectively

2.1 Command Syntax

All Linux commands follow this structure:

```
command [options] [arguments]
```

Example: `ls -l /home`

Part	Example	Description
Command	<code>ls</code>	The program to run
Option	<code>-l</code>	Modifies command behaviour (use a dash -)
Argument	<code>/home</code>	The target file or directory

 **TIP:** Linux is case-sensitive. `ls`, `LS`, and `Ls` are three different commands.

2.2 Essential Commands Reference

Command	Purpose & Example
<code>pwd</code>	Print Working Directory — shows your current location Output: <code>/home/ec2-user</code>
<code>ls</code>	List files and directories in the current directory
<code>ls -l</code>	Long listing — shows permissions, owner, size, and date
<code>ls -a</code>	Show all files including hidden files (starting with <code>.</code>)
<code>ls -lah</code>	Long listing, all files, human-readable sizes — most used combination
<code>cd dir</code>	Change into a directory (e.g., <code>cd Documents</code>)
<code>cd ..</code>	Move one level up to parent directory

Command	Purpose & Example
<code>cd ~</code>	Go to your home directory from anywhere
<code>cd -</code>	Return to the previous directory
<code>cd /</code>	Go to the root directory
<code>clear</code>	Clear the terminal screen (or press Ctrl + L)
<code>history</code>	Show the list of previously executed commands
<code>!!</code>	Repeat the last command
<code>!4</code>	Run command number 4 from history
<code>man ls</code>	Display the manual page for <code>ls</code> (q to quit)
<code>which python3</code>	Show the full path of an executable command
<code>whereis ssh</code>	Show binary, source, and manual page locations
<code>echo Hello</code>	Display text or variable values on screen
<code>echo \$HOME</code>	Display the value of the HOME environment variable
<code>date</code>	Display the current date and time
<code>date +"%d-%m-%Y"</code>	Display date in DD-MM-YYYY format
<code>cal</code>	Display the calendar for the current month
<code>cal 2025</code>	Display the full-year calendar for 2025
<code>uname</code>	Display the operating system name (Linux)
<code>uname -a</code>	Display full system info: kernel, hostname, architecture
<code>hostname</code>	Display the system hostname (e.g., ip-172-31-10-20)

2.3 Common Mistakes

⚠ WARNING: CD Documents — Linux is case-sensitive. Always use lowercase: `cd Documents`

⚠ WARNING: `ls a` — Missing the dash. Correct form: `ls -a`

💡 TIP: Always run `pwd` before deleting files to confirm your location.

Interview Questions

- Q What does `pwd` do?
- Q What is the difference between `cd` and `pwd`?
- Q What does `ls -la` display?
- Q How do you check command history?
- Q What is the purpose of man pages?
- Q What is the difference between `which` and `whereis`?
- Q Why is Linux case-sensitive?

Chapter Summary

In this chapter you learned:

- ✓ Linux command syntax: `command [options] [arguments]`
- ✓ `pwd` shows current directory; `ls` lists files; `cd` changes directory
- ✓ `history` and `!!` help recall and repeat previous commands
- ✓ `man` provides documentation; `which` and `whereis` locate executables
- ✓ `echo` displays text and variables; `date`, `cal`, `uname` provide system info

CHAPTER 3

Working with Files and Directories

Learning Objectives

After completing this chapter, you will be able to:

- Create files and directories using touch and mkdir
- Copy, move, and rename files with cp and mv
- Safely delete files and directories with rm and rmdir
- View file type and metadata using file and stat
- Visualize directory structures with tree

3.1 Command Reference

Command	Purpose & Example
touch file.txt	Create an empty file (or update its timestamp if it exists)
touch a.txt b.txt c.txt	Create multiple files at once
mkdir project	Create a directory called project
mkdir dev test prod	Create multiple directories at once
mkdir -p project/backend/api	Create nested directories (parents created automatically)
rmdir test	Remove an empty directory (fails if directory has content)
cp notes.txt backup.txt	Copy a file (source → destination)
cp notes.txt Documents/	Copy a file into a directory
cp -r project project_bak	Copy a directory recursively (-r required for directories)
mv notes.txt old.txt	Rename a file (move source to destination)
mv notes.txt /tmp	Move a file to /tmp
mv project /home/ec2-user	Move a directory
rm notes.txt	Delete a file permanently
rm -i notes.txt	Interactive delete — prompts before removing
rm -r project	Recursively delete a directory and all its contents
rm -rf project	Force recursive delete — no confirmation (use with caution!)
file notes.txt	Detect and display the type of a file (e.g., ASCII text)
stat notes.txt	Display detailed metadata: size, permissions, owner, timestamps
tree	Show the directory structure visually as a tree
tree /var/www/html	Show the structure of a specific directory

3.2 Real AWS EC2 Scenario — Web App Structure

Create a complete web application directory structure in one command:

```
# Create nested folders in one step
mkdir -p myapp/{logs,config,backup}

# Verify with tree
tree myapp
# myapp
# └── backup
# └── config
# └── logs

# Create config and log files
touch myapp/config/app.conf
touch myapp/logs/app.log

# Backup the config
cp myapp/config/app.conf myapp/backup/

# Rename the log file
mv myapp/logs/app.log myapp/logs/application.log
```

⚠ WARNING: Never run `rm -rf /` — this destroys the entire Linux operating system.

💡 TIP: Always run `pwd` before using `rm` to confirm you are in the correct directory.

Interview Questions

- Q What does `touch` do?
- Q What is the difference between `mkdir` and `mkdir -p`?
- Q What is the difference between `cp` and `mv`?
- Q What is the difference between `rm` and `rmdir`?
- Q What does `rm -rf` do and why is it dangerous?
- Q How do you copy a directory?

Chapter Summary

In this chapter you learned:

- ✓ `touch` creates empty files; `mkdir` creates directories
- ✓ `mkdir -p` creates full nested directory trees in one command
- ✓ `cp` copies files; `mv` moves or renames files
- ✓ `rm` deletes files; `rm -r` deletes directories recursively
- ✓ `file` detects file type; `stat` shows detailed metadata
- ✓ `tree` visualises directory structures clearly

CHAPTER 4

Viewing and Editing Files

Learning Objectives

After completing this chapter, you will be able to:

- View file contents with cat, tac, more, and less
- Read the beginning and end of files with head and tail
- Monitor log files in real time with tail -f
- Edit files with the beginner-friendly Nano editor
- Edit files with the powerful Vim editor
- Work with configuration files on AWS EC2

4.1 Viewing Commands

Command	Purpose & Example
cat file.txt	Display entire file contents — best for small files
cat -n file.txt	Display file contents with line numbers
cat file1 file2	Display multiple files concatenated in sequence
tac file.txt	Display file in reverse order (last line first)
more /etc/passwd	View large file page by page (Space = next, q = quit)
less /etc/passwd	Improved pager: scroll up/down, search with /word, q = quit
head file.txt	Display first 10 lines of a file
head -5 file.txt	Display first 5 lines
tail file.txt	Display last 10 lines of a file
tail -20 file.txt	Display last 20 lines
tail -f app.log	Monitor a file in real time — new lines appear immediately (Ctrl+C to stop)

4.2 Nano Editor — Beginner Friendly

Nano is the recommended editor for beginners. It shows keyboard shortcuts at the bottom.

```
# Open or create a file in Nano
nano app.conf

# Add content, then:
Ctrl + O → Save (Write Out)
Enter   → Confirm filename
Ctrl + X → Exit
Ctrl + K → Cut line
Ctrl + U → Paste line
Ctrl + W → Search
```

4.3 Vim Editor — Professional Standard

Vim is used by Linux administrators and DevOps engineers worldwide. It has two primary modes:

Mode	Purpose	How to Enter
Normal (Command)	Navigate, save, quit	Press Esc
Insert	Type and edit text	Press i

Essential Vim commands:

Command	Action
i	Enter Insert mode (start typing)
Esc	Return to Normal mode
:w	Save the file
:q	Quit Vim
:wq	Save and quit
:q!	Quit without saving (force)
dd	Delete current line
yy	Copy (yank) current line
p	Paste below cursor
/word	Search for 'word' in file

4.4 Real AWS Troubleshooting Workflow

When a web application is failing:

```
# Step 1 — Check real-time logs
tail -f /var/log/messages

# Step 2 — View configuration
cat /etc/httpd/conf/httpd.conf

# Step 3 — Edit configuration
sudo nano /etc/httpd/conf/httpd.conf

# Step 4 — Restart service
sudo systemctl restart httpd
```

 **TIP:** Use `less` for large log files — it allows scrolling up and down without loading the entire file into memory.

Interview Questions

- Q What is the difference between `cat` and `less`?
- Q What is the difference between `more` and `less`?
- Q What does `tail -f` do and when is it used?
- Q What are the two main modes in Vim?
- Q How do you save and quit Vim?

Chapter Summary

In this chapter you learned:

- ✓ `cat` displays files; `tac` shows them in reverse
- ✓ `less` is preferred over `more` for large files — supports bidirectional scrolling
- ✓ `head` shows the beginning; `tail` shows the end of a file
- ✓ `tail -f` monitors live log output in real time
- ✓ Nano is beginner-friendly; Vim is powerful and industry-standard

CHAPTER 5

Searching and Text Processing

Learning Objectives

After completing this chapter, you will be able to:

- Search for files and directories using `find` and `locate`
- Search for text patterns using `grep` and `egrep`
- Filter, count, and sort text with `wc`, `sort`, and `uniq`
- Extract fields from structured text with `cut` and `awk`
- Translate and replace characters with `tr`
- Search and replace text in files using `sed`
- Chain commands together using pipes (`|`)

5.1 Command Reference

Command	Purpose & Example
<code>find /home -name notes.txt</code>	Search for a file by name in real time
<code>find . -name '*.txt'</code>	Find all .txt files in the current directory tree
<code>find / -size +100M</code>	Find files larger than 100 MB
<code>find . -name '*.tmp' -delete</code>	Find and delete all .tmp files
<code>locate passwd</code>	Fast database search for files matching 'passwd'
<code>sudo updatedb</code>	Refresh the locate database (run after creating new files)
<code>grep root /etc/passwd</code>	Search for lines containing 'root' in /etc/passwd
<code>grep -i linux notes.txt</code>	Case-insensitive search
<code>grep -n root /etc/passwd</code>	Show matching lines with line numbers
<code>grep -r 'error' /var/log</code>	Recursive search through all files in /var/log
<code>grep -c root /etc/passwd</code>	Count the number of matching lines
<code>grep -v root users.txt</code>	Show all lines NOT matching (invert)
<code>egrep "root ec2-user" /etc/passwd</code>	Extended grep — matches root OR ec2-user
<code>wc notes.txt</code>	Count lines, words, and characters in a file
<code>wc -l notes.txt</code>	Count lines only
<code>sort users.txt</code>	Sort file contents alphabetically
<code>sort -r users.txt</code>	Sort in reverse order
<code>sort -n marks.txt</code>	Numeric sort
<code>uniq skills.txt</code>	Remove consecutive duplicate lines

Command	Purpose & Example
<code>uniq -c skills.txt</code>	Count occurrences of each unique line
<code>sort file uniq</code>	Correct way to remove all duplicates
<code>cut -d ":" -f1 /etc/passwd</code>	Extract first field (username) using : as delimiter
<code>cut -d ":" -f3 employees.txt</code>	Extract third field (salary)
<code>echo 'linux' tr a-z A-Z</code>	Convert lowercase to uppercase: outputs LINUX
<code>echo 'linux-admin' tr '-' '_'</code>	Replace - with _: outputs linux_admin
<code>awk '{print \$1}' file.txt</code>	Print the first column (space-delimited)
<code>awk -F ':' '{print \$1}' /etc/passwd</code>	Print usernames using : as field separator
<code>sed 's/Linux/AWS/' notes.txt</code>	Replace first occurrence of Linux with AWS
<code>sed 's/Linux/AWS/g' notes.txt</code>	Replace all occurrences
<code>sed -i 's/8080/80/g' app.conf</code>	Edit file directly (in-place replacement)
<code>cmd1 cmd2</code>	Pipe: send output of cmd1 as input to cmd2
<code>ps -ef grep ssh</code>	Find running SSH processes
<code>cat /etc/passwd wc -l</code>	Count users in /etc/passwd

5.2 Real AWS EC2 Scenario — Analysing Failed Logins

```
# Find all failed SSH login attempts
grep "Failed password" /var/log/secure

# Count the number of failures
grep "Failed password" /var/log/secure | wc -l

# Extract unique attacking usernames
cut -d ' ' -f1 failed_users.txt | sort | uniq
```

 **TIP:** Always sort before using uniq. The uniq command only removes consecutive duplicate lines.

 **WARNING:** Before using sed -i, always back up the file: `cp config.conf config.conf.bak`

Interview Questions

- Q What is the difference between find and locate?
- Q What does grep -v do?
- Q What is a pipe (|) in Linux?
- Q What is the difference between grep and egrep?
- Q What does awk do?
- Q What is sed used for?

Chapter Summary

In this chapter you learned:

- ✓ find performs real-time file searches; locate uses a fast database
- ✓ grep is the most important text search command
- ✓ wc counts lines/words/characters; sort sorts text; uniq removes duplicates
- ✓ cut extracts specific fields; awk processes structured text by columns
- ✓ tr translates/replaces characters; sed performs stream editing and replacement
- ✓ Pipes (|) connect commands together into powerful processing chains

CHAPTER 6

Users and Groups Management

Learning Objectives

After completing this chapter, you will be able to:

- Understand Linux user types (root, regular, system)
- Identify important user-related files (/etc/passwd, /etc/shadow, /etc/group)
- Create, modify, and delete user accounts
- Create and manage groups
- Set and change user passwords
- Use sudo and su to elevate privileges

6.1 User Types

Type	Examples	UID Range
Root (Superuser)	root	0
Regular Users	john, sam, ec2-user, ubuntu	1000+
System Users	apache, nginx, mysql, sshd	< 1000

6.2 Important User Files

File	Purpose	View Command
/etc/passwd	Stores user account information (username, UID, home, shell)	cat /etc/passwd
/etc/shadow	Stores encrypted passwords (root access only)	sudo cat /etc/shadow
/etc/group	Stores group information and membership	cat /etc/group

6.3 Command Reference

Command	Purpose & Example
whoami	Display the currently logged-in user
id	Show UID, GID, and all group memberships for current user
id john	Show UID, GID, and groups for user 'john'
who	Show all currently logged-in users with terminal info

Command	Purpose & Example
<code>users</code>	Show just the usernames of logged-in users
<code>groups</code>	Show group membership for the current user
<code>groups john</code>	Show group membership for 'john'
<code>sudo useradd -m john</code>	Create user 'john' with a home directory
<code>sudo passwd john</code>	Set or change the password for 'john'
<code>sudo usermod -aG developers john</code>	Add 'john' to the 'developers' group (-a = append, -G = supplementary group)
<code>sudo usermod -d /data/john john</code>	Change home directory for 'john'
<code>sudo userdel john</code>	Delete user 'john' (home directory kept)
<code>sudo userdel -r john</code>	Delete user 'john' AND their home directory
<code>sudo groupadd developers</code>	Create a new group called 'developers'
<code>sudo groupdel developers</code>	Delete the 'developers' group
<code>sudo gpasswd -a john developers</code>	Add 'john' to group 'developers'
<code>sudo gpasswd -d john developers</code>	Remove 'john' from group 'developers'
<code>su john</code>	Switch to user 'john' (keeps current environment)
<code>su - john</code>	Switch to 'john' with a full login shell (loads their environment)
<code>sudo command</code>	Execute a command with root privileges (recommended over logging in as root)

6.4 Complete User Management Workflow

```
# 1 — Create a new group
sudo groupadd developers

# 2 — Create a user with a home directory
sudo useradd -m john

# 3 — Set a password
sudo passwd john

# 4 — Add user to group
sudo usermod -aG developers john

# 5 — Verify
groups john
# Output: john developers
```

⚠ WARNING: Always use -aG (append) with usermod. Using -G alone removes the user from all existing groups.

Interview Questions

- Q What is the root user?
- Q What information is stored in `/etc/passwd` vs `/etc/shadow`?
- Q What is the difference between `su` and `sudo`?
- Q What is the difference between `useradd` and `usermod`?
- Q How do you add a user to a group without removing existing groups?

Chapter Summary

In this chapter you learned:

- ✓ Linux has three user types: root (UID 0), regular users (UID 1000+), and system users
- ✓ `/etc/passwd` stores account info; `/etc/shadow` stores encrypted passwords
- ✓ `useradd -m` creates users with home directories; `passwd` sets passwords
- ✓ `usermod -aG` adds users to groups without removing existing memberships
- ✓ `sudo` is preferred over logging in as root for security

CHAPTER 7

File Permissions and Ownership

Learning Objectives

After completing this chapter, you will be able to:

- Read and interpret Linux permission strings
- Apply permissions using chmod (numeric and symbolic)
- Change file ownership with chown and group with chgrp
- Understand default permissions and umask
- Explain SUID, SGID, and Sticky Bit

7.1 Understanding Permissions

Run `ls -l` to view permissions:

```
-rwxr-xr-- 1 ec2-user developers 1250 Jun 22 script.sh
```

Characters	Category	Permissions
- (1st char)	File type (- = file, d = directory, l = link)	
rwx (2-4)	Owner permissions	Read + Write + Execute
r-x (5-7)	Group permissions	Read + Execute
r-- (8-10)	Others permissions	Read only

7.2 Numeric Permission Values

Value	Symbolic	Meaning
7	rwx	Read + Write + Execute (full access)
6	rw-	Read + Write
5	r-x	Read + Execute
4	r--	Read only
0	---	No permissions

7.3 Common Permission Combinations

Value	Use Case
777 (rwxrwxrwx)	Full access for everyone — avoid in production!
755 (rwxr-xr-x)	Executable scripts and programs
700 (rwx-----)	Private scripts — owner only
644 (rw-r--r--)	Standard files (configs, documents)
600 (rw-----)	Sensitive files (SSH keys, passwords)

7.4 Command Reference

Command	Purpose & Example
chmod 755 script.sh	Set permissions to rwxr-xr-x (numeric method)
chmod 644 notes.txt	Set permissions to rw-r--r--
chmod 600 secrets.txt	Set permissions to rw----- (private)
chmod +x script.sh	Add execute permission for owner (symbolic method)
chmod -x script.sh	Remove execute permission
chmod g+w file.txt	Add write permission for group
chmod o-r file.txt	Remove read permission from others
chmod -R 755 project/	Apply permissions recursively to entire directory
chown john notes.txt	Change file owner to 'john'
chown john:dev notes.txt	Change owner to 'john' and group to 'dev'
sudo chown -R ec2-user /var/www/html	Recursively change ownership of web root
chgrp developers notes.txt	Change group ownership only
umask	Display current default permission mask
umask 022	Set default mask (new files = 644, dirs = 755)
umask 077	Strict mask for sensitive environments (new files = 600)
chmod u+s script.sh	Set SUID — file runs with owner's permissions
chmod g+s project/	Set SGID — new files inherit directory group
chmod +t shared/	Set Sticky Bit — users can only delete their own files

7.5 Special Permissions

Permission	Symbol	Numeric	Effect
SUID	s on owner execute	4xxx (e.g., 4755)	File runs with the file owner's permissions
SGID	s on group execute	2xxx (e.g., 2755)	Files in directory inherit group ownership
Sticky Bit	t on others execute	1xxx (e.g., 1777)	Only file owner can delete their own files (used in /tmp)

⚠ WARNING: `chmod 777` is a serious security risk. Never use it on production systems.

💡 TIP: AWS SSH key files must have permission 600 or SSH will reject them: `chmod 600 mykey.pem`

Interview Questions

- Q What do `rxw` permissions mean?
- Q Explain `chmod 755`
- Q What is the difference between `chown` and `chgrp`?
- Q What is `umask`?
- Q What is SUID and when is it used?
- Q What is the Sticky Bit?

Chapter Summary

In this chapter you learned:

- ✓ Linux permissions have three categories: owner (u), group (g), and others (o)
- ✓ Numeric method: `r=4`, `w=2`, `x=1`; add them to get permission value
- ✓ `chmod` changes permissions; `chown` changes ownership; `chgrp` changes group
- ✓ `umask` controls default permissions for newly created files
- ✓ SUID, SGID, and Sticky Bit provide special security controls

CHAPTER 8

Process Management

Learning Objectives

After completing this chapter, you will be able to:

- Understand Linux processes and process IDs (PIDs)
- View running processes with `ps`, `top`, and `htop`
- Monitor system performance and memory usage
- Manage foreground and background jobs
- Terminate processes safely using `kill`, `pkill`, and `killall`
- Run persistent background processes with `nohup`

8.1 Process Terminology

Term	Definition
Process	A running instance of a program
PID	Process ID — unique number assigned to each process
PPID	Parent Process ID — the process that started this one
Daemon	A background service process (e.g., <code>sshd</code> , <code>httpd</code>)
Foreground	Process running in and blocking the current terminal
Background	Process running independently (use <code>&</code>)

8.2 Command Reference

Command	Purpose & Example
<code>ps</code>	Show processes running in the current terminal session
<code>ps -ef</code>	Show ALL running processes on the system
<code>ps aux</code>	Detailed view with CPU%, memory%, and owner
<code>ps -ef grep httpd</code>	Find if Apache is running
<code>top</code>	Real-time system monitor — CPU, memory, processes (q to quit)
<code>htop</code>	Enhanced top with colour and mouse support (F10 to quit)
<code>pgrep sshd</code>	Find the PID(s) of processes named <code>sshd</code>
<code>pidof sshd</code>	Display the PID of a running process by name
<code>kill PID</code>	Gracefully terminate a process (SIGTERM signal 15)
<code>kill -9 PID</code>	Force-kill a process immediately (SIGKILL signal 9)

Command	Purpose & Example
kill -15 PID	Same as kill PID — graceful shutdown
kill -1 PID	Send HUP signal — reload configuration without stopping
pkill firefox	Kill processes by name (like kill but uses name)
killall httpd	Terminate ALL instances of a process by name
sleep 300 &	Run a command in the background with &
jobs	List all background and suspended jobs
fg %1	Bring job number 1 to the foreground
bg	Resume a suspended process in the background
nohup python app.py &	Run a command that survives logout (output → nohup.out)
nice -n 10 cmd	Start a process with lower CPU priority (range: -20 to 19)
sudo renice 5 -p PID	Change CPU priority of an already-running process
uptime	Show system uptime and load averages (1, 5, 15 minutes)
free -h	Show memory usage in human-readable format

8.3 Real AWS EC2 Scenario — Application is Slow

```
# Step 1 — View all processes sorted by resource usage
ps aux

# Step 2 — Real-time monitoring
top

# Step 3 — Find the specific process
pgrep python

# Step 4 — Check memory
free -h

# Step 5 — Kill a hung process
kill -9 <PID>

# Step 6 — Restart application as persistent background process
nohup python app.py &

# Step 7 — Verify
ps -ef | grep python
```

⚠ WARNING: Always try kill (SIGTERM) before kill -9 (SIGKILL). Force-killing may corrupt data or leave locks.

💡 TIP: Use nohup for any process that must survive terminal closure or SSH disconnection.

Interview Questions

- Q What is a process?
- Q What is the difference between kill and kill -9?
- Q What is the difference between kill and pkill?
- Q What does nohup do and why is it important in AWS?
- Q What do fg and bg do?

Chapter Summary

In this chapter you learned:

- ✓ Every running program is a process with a unique PID
- ✓ ps -ef shows all processes; ps aux shows CPU and memory usage
- ✓ top and htop provide real-time performance monitoring
- ✓ kill uses PID; pkill uses name; killall terminates all named instances
- ✓ nohup keeps processes running after logout — essential for AWS workloads
- ✓ & runs commands in background; jobs lists them; fg/bg control them

CHAPTER 9

Package Management and Software Installation

Learning Objectives

After completing this chapter, you will be able to:

- Understand RPM and DEB package formats
- Install, update, and remove packages using yum and dnf
- Manage packages with rpm directly
- Use apt and dpkg on Ubuntu/Debian systems
- Manage software repositories

9.1 Package Managers Overview

Distribution	Package Format	Package Manager
Amazon Linux 2023	RPM	dnf (primary)
Amazon Linux 2	RPM	yum
RHEL / CentOS	RPM	dnf / yum
Ubuntu / Debian	DEB	apt / dpkg

9.2 DNF Commands (Amazon Linux 2023)

Command	Purpose
<code>sudo dnf update -y</code>	Update all installed packages on the system
<code>sudo dnf install httpd -y</code>	Install Apache web server
<code>dnf search nginx</code>	Search for Nginx in available repositories
<code>dnf info nginx</code>	Show detailed package information
<code>sudo dnf remove nginx</code>	Remove a package
<code>dnf list installed</code>	List all currently installed packages
<code>dnf check-update</code>	Show available updates without installing
<code>sudo dnf clean all</code>	Clean the local package cache
<code>dnf repolist</code>	List all enabled repositories

9.3 YUM Commands (Amazon Linux 2)

Command	Purpose
<code>sudo yum install httpd -y</code>	Install Apache
<code>sudo yum update -y</code>	Update all packages
<code>sudo yum remove httpd</code>	Remove Apache
<code>yum list installed</code>	List installed packages
<code>yum search nginx</code>	Search repositories
<code>yum info httpd</code>	Show package details

9.4 RPM Direct Package Management

Command	Purpose
<code>rpm -qa</code>	List all installed RPM packages
<code>rpm -qa grep httpd</code>	Check if httpd is installed
<code>rpm -qi httpd</code>	Detailed package info (version, size, description)
<code>rpm -ql httpd</code>	List all files installed by the package
<code>sudo rpm -ivh package.rpm</code>	Install an RPM file (-i install, -v verbose, -h progress)
<code>sudo rpm -Uvh package.rpm</code>	Upgrade an RPM file
<code>sudo rpm -e package_name</code>	Remove an installed RPM package

9.5 APT Commands (Ubuntu EC2)

Command	Purpose
<code>sudo apt update</code>	Refresh repository metadata (always run before installing)
<code>sudo apt upgrade -y</code>	Upgrade all installed packages
<code>sudo apt install nginx -y</code>	Install Nginx
<code>sudo apt remove nginx</code>	Remove package (keeps config files)
<code>sudo apt purge nginx</code>	Remove package AND configuration files
<code>apt list --installed</code>	List installed packages
<code>apt search nginx</code>	Search available packages

9.6 Real AWS EC2 Workflow — Deploy Apache

```
# Amazon Linux 2023
sudo dnf update -y
sudo dnf install httpd -y
sudo systemctl start httpd
sudo systemctl enable httpd
systemctl status httpd
rpm -qa | grep httpd
```

⚠ WARNING: Always run `sudo apt update` before `sudo apt install` on Ubuntu. Installing without updating may pull outdated versions.

Interview Questions

- Q What is the difference between yum and dnf?
- Q What is the difference between apt and dpkg?
- Q How do you update all packages on Amazon Linux?
- Q What does `rpm -qa` do?
- Q What is a package repository?

Chapter Summary

In this chapter you learned:

- ✓ Amazon Linux 2023 uses dnf; Amazon Linux 2 uses yum; Ubuntu uses apt
- ✓ `rpm -qa` lists installed packages; `rpm -qi` shows package details
- ✓ Always update repository metadata before installing new packages
- ✓ Use `-y` flag to skip confirmation prompts in automation scripts

CHAPTER 10

Service Management and Systemctl

Learning Objectives

After completing this chapter, you will be able to:

- Understand systemd and its role as PID 1
- Start, stop, restart, and reload services
- Enable services to auto-start at boot
- Check and interpret service status
- Read logs with journalctl
- Troubleshoot failed services systematically

10.1 systemd Overview

Modern Linux distributions use systemd as the system and service manager. It is always PID 1 — the first process started at boot.

```
# Verify systemd is PID 1
ps -p 1
# Output: PID 1  systemd
```

10.2 Service States

State	Meaning
active (running)	Service is running normally
inactive	Service is stopped
failed	Service crashed or exited with an error
activating	Service is in the process of starting
deactivating	Service is in the process of stopping
enabled	Service will auto-start at next boot
disabled	Service will NOT auto-start at boot

10.3 Command Reference

Command	Purpose
<code>systemctl status httpd</code>	Show current status and recent log lines for httpd
<code>sudo systemctl start httpd</code>	Start the httpd service
<code>sudo systemctl stop httpd</code>	Stop the httpd service
<code>sudo systemctl restart httpd</code>	Stop then start (brief downtime)
<code>sudo systemctl reload nginx</code>	Reload config without stopping service (no downtime)
<code>sudo systemctl enable httpd</code>	Auto-start httpd on every boot
<code>sudo systemctl disable httpd</code>	Prevent httpd from starting at boot
<code>systemctl is-enabled httpd</code>	Check whether httpd is set to start at boot
<code>systemctl list-units --type=service</code>	List all currently running services
<code>systemctl list-unit-files --type=service</code>	List all installed services and their boot status
<code>sudo systemctl daemon-reload</code>	Reload systemd after editing unit files
<code>journalctl</code>	View all systemd logs
<code>journalctl -n 20</code>	View last 20 log lines
<code>journalctl -f</code>	Follow logs in real time (like tail -f)
<code>journalctl -u httpd</code>	Show logs for the httpd service only
<code>journalctl --since today</code>	Show all logs from today
<code>journalctl --since '1 hour ago'</code>	Show logs from the past hour
<code>journalctl -p err</code>	Show only error-level logs

10.4 Service Troubleshooting Workflow

```
# Step 1 — Check service status
systemctl status httpd
```

```
# Step 2 — Read detailed logs
journalctl -u httpd
```

```
# Step 3 — Restart the service
sudo systemctl restart httpd
```

```
# Step 4 — Check listening port
ss -tulnp | grep 80
```

```
# Step 5 — Verify process
ps -ef | grep httpd
```

⚠ WARNING: Starting a service does NOT automatically enable it at boot. Always run `systemctl enable` after `systemctl start` for production services.

💡 TIP: `journalctl -xe` provides the most detailed view when troubleshooting service startup failures.

Interview Questions

- Q What is systemd?
- Q What is the difference between start and enable?
- Q What is the difference between restart and reload?
- Q What is journalctl?
- Q What is PID 1?

Chapter Summary

In this chapter you learned:

- ✓ systemd is the modern system and service manager; it is always PID 1
- ✓ start/stop control running state; enable/disable control boot behaviour
- ✓ reload reloads config with no downtime; restart involves a brief outage
- ✓ journalctl reads systemd logs; journalctl -f follows them in real time

CHAPTER 11

Networking Fundamentals & Troubleshooting

Learning Objectives

After completing this chapter, you will be able to:

- Check IP addresses and routing information
- Test connectivity with ping and traceroute
- Identify open and listening ports
- Troubleshoot DNS with nslookup and dig
- Transfer data and test HTTP endpoints with curl
- Troubleshoot common AWS EC2 network issues

11.1 Key Networking Concepts

Term	Definition
IP Address	Unique identifier for a device on a network
DNS	Translates domain names (google.com) to IP addresses
Port	A numbered endpoint for network communication
Gateway	The exit point to reach external networks
Subnet	A segment or division of a network

11.2 Common Port Numbers

Service	Port	Protocol
SSH	22	TCP
HTTP	80	TCP
HTTPS	443	TCP
DNS	53	TCP/UDP
FTP	21	TCP
MySQL	3306	TCP
PostgreSQL	5432	TCP

11.3 Command Reference

Command	Purpose
hostname	Display the system hostname
hostname -f	Display the fully qualified domain name (FQDN)
hostnamectl	Show hostname and OS/kernel details
ip addr	Show all network interfaces and IP addresses (use ip a for short)
ip link	Show network interface names and states
ip route	Show the routing table (default gateway)
ifconfig	Legacy — show network info (install net-tools if missing)
ping -c 4 google.com	Send 4 ICMP packets to test connectivity
ping -c 4 8.8.8.8	Ping Google's DNS to test internet without DNS dependency
ss -tulnp	Show all listening ports with process info (-t TCP, -u UDP, -l listening, -n numeric, -p process)
netstat -tulnp	Legacy equivalent of ss -tulnp (install net-tools)
traceroute amazon.com	Show network path (hop by hop) to a destination
tracepath google.com	Alternative to traceroute — no root required
nslookup google.com	Basic DNS lookup — resolve a domain name to IP
nslookup google.com 8.8.8.8	DNS lookup using a specific DNS server
dig google.com	Advanced DNS lookup with detailed output
dig +short amazon.com	Quick IP-only DNS lookup result
dig amazon.com MX	Query mail server (MX) DNS records
host google.com	Simple one-line DNS lookup
curl https://google.com	Fetch a webpage and display its HTML content
curl -I https://google.com	Fetch only HTTP response headers (status code)
curl http://localhost	Test if a local web server is responding
curl -O https://example.com/file.zip	Download a file (saves with original filename)
wget https://example.com/file.zip	Download a file (alternative to curl -O)
arp -a	Display the ARP table (MAC addresses on local network)
ip route	Show routing table (preferred over legacy route -n)

11.4 Real AWS Troubleshooting Scenarios

Scenario 1 — Website Not Accessible

```
# Check if the service is running
systemctl status httpd

# Confirm port 80 is listening
ss -tulnp | grep 80

# Test locally on the server
curl localhost

# Test internet connectivity
ping -c 4 google.com

# → If all above pass: check AWS Security Group inbound rules
```

Scenario 2 — DNS Not Working

```
# Test raw IP connectivity first
ping 8.8.8.8      # Works? Internet is OK

# Test DNS resolution
nslookup google.com # Fails? DNS issue confirmed

# Check DNS resolver configuration
cat /etc/resolv.conf
```

Interview Questions

- Q What is DNS?
- Q What is the difference between ping and traceroute?
- Q What is the difference between ss and netstat?
- Q What is the difference between curl and wget?
- Q How do you check which process is listening on port 80?

Chapter Summary

In this chapter you learned:

- ✓ ip addr shows IP addresses; ip route shows the routing table
- ✓ ping tests connectivity; traceroute shows the network path
- ✓ ss -tulnp shows all listening ports and associated processes
- ✓ nslookup performs basic DNS lookups; dig provides advanced analysis
- ✓ curl tests HTTP endpoints and downloads files; wget downloads files

CHAPTER 12

Disk Management and Storage

Learning Objectives

After completing this chapter, you will be able to:

- Check filesystem and directory usage with `df` and `du`
- List and identify block devices with `lsblk` and `blkid`
- Mount and unmount filesystems
- Create partitions with `fdisk` and `parted`
- Format devices with `mkfs`
- Configure permanent mounts via `/etc/fstab`
- Manage AWS EBS volumes through their full lifecycle

12.1 Storage Concepts

Concept	Definition	Example
Disk	Physical or virtual storage device	<code>/dev/xvda</code> , <code>/dev/nvme0n1</code>
Partition	A logical division of a disk	<code>/dev/xvda1</code> , <code>/dev/nvme0n1p1</code>
Filesystem	Organises data on a partition	<code>ext4</code> , <code>xf</code> s, <code>btrf</code> s
Mount Point	Directory where a filesystem is attached	<code>/</code> , <code>/home</code> , <code>/data</code>

12.2 Command Reference

Command	Purpose
<code>df -h</code>	Show filesystem disk usage in human-readable format
<code>df -Th</code>	Show usage with filesystem type (<code>xf</code> s, <code>ext4</code> , etc.)
<code>df -i</code>	Show inode usage (inode exhaustion causes 'no space' errors too)
<code>du -sh /var/log</code>	Show total size of <code>/var/log</code> directory
<code>du -sh *</code>	Show size of every item in the current directory
<code>du -sh /var/log/*</code>	Show size of each item inside <code>/var/log</code>
<code>lsblk</code>	List all block devices and their partitions in tree view
<code>lsblk -f</code>	Show filesystem type and mount points alongside devices
<code>sudo blkid</code>	Show device UUIDs and filesystem types
<code>mount</code>	Show all currently mounted filesystems

Command	Purpose
<code>sudo mount /dev/xvdf1 /data</code>	Mount a device to the /data directory
<code>sudo umount /data</code>	Unmount the /data filesystem
<code>sudo fdisk -l</code>	List all disks and partitions
<code>sudo fdisk /dev/xvdf</code>	Interactive partition tool for MBR disks
<code>sudo parted -l</code>	List partitions (supports GPT and disks >2TB)
<code>sudo mkfs.xfs /dev/xvdf1</code>	Format partition with XFS filesystem
<code>sudo mkfs.ext4 /dev/xvdf1</code>	Format partition with EXT4 filesystem
<code>sudo fsck /dev/xvdf1</code>	Check and repair a filesystem
<code>mountpoint /data</code>	Verify whether /data is a mount point
<code>findmnt</code>	Display all mounted filesystems in tree format
<code>findmnt /data</code>	Show details for the /data mount
<code>cat /etc/fstab</code>	View permanent mount configuration
<code>sudo mount -a</code>	Test /etc/fstab without rebooting

12.3 AWS EBS Complete Workflow

```

# Step 1 — Attach EBS volume in AWS Console
# EC2 → Volumes → Attach Volume

# Step 2 — Verify device appeared
lsblk

# Step 3 — Create a partition
sudo fdisk /dev/xvdf
# → n (new), p (primary), w (save)

# Step 4 — Format with XFS
sudo mkfs.xfs /dev/xvdf1

# Step 5 — Create mount point
sudo mkdir /data

# Step 6 — Mount
sudo mount /dev/xvdf1 /data

# Step 7 — Get UUID for permanent mount
sudo blkid

# Step 8 — Add to /etc/fstab
# UUID=abc-123 /data xfs defaults 0 0

# Step 9 — Test fstab
sudo mount -a

# Step 10 — Verify
df -h

```

⚠ WARNING: Always verify with `lsblk` before formatting. Running `mkfs` on the wrong device destroys data permanently.

💡 TIP: Inode exhaustion can cause 'No space left on device' even when `df -h` shows free space. Check `df -i`.

Interview Questions

- Q What is the difference between `df` and `du`?
- Q What is a mount point?
- Q What is the purpose of `/etc/fstab`?
- Q What is UUID and why is it used in `/etc/fstab`?
- Q How do you troubleshoot a full disk?

Chapter Summary

In this chapter you learned:

- ✓ `df` shows filesystem usage; `du` shows directory/file sizes
- ✓ `lsblk` lists all block devices; `blkid` shows UUIDs
- ✓ `mount` attaches filesystems; `umount` detaches them
- ✓ `fdisk` creates MBR partitions; `parted` supports GPT and large disks
- ✓ `/etc/fstab` enables permanent mounts that survive reboots

CHAPTER 13

Log Management and System Monitoring

Learning Objectives

After completing this chapter, you will be able to:

- Understand and navigate important Linux log locations
- Monitor system performance with `top`, `htop`, and `vmstat`
- Check memory usage with `free`
- Analyse disk performance with `iostat`
- View historical statistics with `sar`
- Troubleshoot performance issues on AWS EC2

13.1 Important Log Locations

Log File	Content
<code>/var/log/messages</code>	General system-wide log messages
<code>/var/log/secure</code>	Authentication and security events
<code>/var/log/cron</code>	Cron job execution logs
<code>/var/log/dmesg</code>	Kernel boot and hardware messages
<code>/var/log/httpd/access_log</code>	Apache web server access log
<code>/var/log/httpd/error_log</code>	Apache web server error log

13.2 Command Reference

Command	Purpose
<code>journalctl -n 20</code>	Show last 20 lines of systemd logs
<code>journalctl -f</code>	Follow systemd logs in real time
<code>journalctl -u httpd</code>	Show logs for the httpd service only
<code>journalctl -p err</code>	Show only error-level messages
<code>journalctl --since today</code>	Show all of today's logs
<code>dmesg tail</code>	Show last kernel messages (hardware, boot events)
<code>dmesg grep nvme</code>	Check kernel messages for NVMe storage devices
<code>tail -f /var/log/messages</code>	Monitor system log in real time
<code>tail -f /var/log/httpd/access_log</code>	Monitor Apache access log live

Command	Purpose
<code>watch -n 2 df -h</code>	Repeat <code>df -h</code> every 2 seconds
<code>free -h</code>	Show memory usage (total, used, free, buffers, cache)
<code>vmstat 2 5</code>	Show CPU, memory, and process stats (5 reports, 2 sec apart)
<code>top</code>	Real-time process and resource monitor (q = quit)
<code>htop</code>	Enhanced top with colour display (F10 = quit)
<code>iostat -x</code>	Extended disk I/O statistics (install <code>sysstat</code>)
<code>iostat -x 2</code>	Refresh disk I/O stats every 2 seconds
<code>sar -u</code>	Show CPU usage history (install <code>sysstat</code>)
<code>sar -r</code>	Show memory usage history
<code>sar -d</code>	Show disk activity history
<code>sar -n DEV</code>	Show network activity history
<code>uptime</code>	Show system uptime and 1/5/15-minute load averages
<code>ps aux --sort=-%cpu head</code>	Find processes consuming the most CPU
<code>ps aux --sort=-%mem head</code>	Find processes consuming the most memory

13.3 Real AWS EC2 Scenarios

Scenario 1 — Server Running Slow

```
# Check load average
uptime

# Find high-CPU processes
top

# Check available memory
free -h

# Check disk I/O performance
iostat -x
```

Scenario 2 — Memory Exhaustion

```
# Check overall memory
free -h

# Find top memory-consuming processes
ps aux --sort=-%mem | head
```

Interview Questions

- Q What is the difference between top and htop?
- Q What does load average mean?
- Q What does iostat measure?
- Q How do you monitor logs in real time?
- Q How do you troubleshoot a slow EC2 instance?

Chapter Summary

In this chapter you learned:

- ✓ journalctl is the primary log viewer for systemd-based systems
- ✓ top and htop provide real-time CPU, memory, and process monitoring
- ✓ free -h shows memory; vmstat shows CPU, memory, and process activity
- ✓ iostat analyses disk performance; sar records historical statistics
- ✓ uptime load averages (1/5/15 min) indicate sustained CPU demand

CHAPTER 14

Bash Scripting Fundamentals

Learning Objectives

After completing this chapter, you will be able to:

- Create and execute shell scripts with a shebang line
- Use variables, read user input, and work with arguments
- Implement conditional logic with if / elif / else
- Repeat actions with for and while loops
- Organise reusable code into functions
- Schedule automatic tasks with cron
- Build practical automation scripts for AWS EC2

14.1 First Bash Script

```
#!/bin/bash
# The shebang (!) tells Linux to use Bash to run this file

echo "Hello AWS re/Start Students"

# Save, give execute permission, then run:
chmod +x hello.sh
./hello.sh
```

14.2 Variables

```
#!/bin/bash

NAME='John'          # No spaces around =
COURSE='AWS re/Start'

echo "$NAME is enrolled in $COURSE"
# Output: John is enrolled in AWS re/Start

# System variables
echo $USER # Current username
echo $HOME # Home directory path
echo $SHELL # Current shell path
```

14.3 User Input and Arguments

```
#!/bin/bash

# Accept user input
echo "Enter your name:"
read NAME
echo "Welcome $NAME"

# Accept command-line arguments
# Run as: ./script.sh AWS Restart
echo "Script: $0" # Script name
echo "Arg 1: $1" # AWS
echo "Arg 2: $2" # Restart
echo "Total: $# " # Number of args
```

14.4 Conditional Logic

Operator	Meaning	Example
-eq	Equal	[\$NUM -eq 10]
-ne	Not equal	[\$NUM -ne 0]
-gt	Greater than	[\$AGE -gt 18]
-lt	Less than	[\$MARKS -lt 50]
-ge	Greater or equal	[\$SCORE -ge 60]
-le	Less or equal	[\$COUNT -le 5]
-f	File exists	[-f /etc/passwd]
-d	Directory exists	[-d /var/log]

```
#!/bin/bash

MARKS=75

if [ $MARKS -ge 90 ]; then
    echo "Grade A"
elif [ $MARKS -ge 70 ]; then
    echo "Grade B"
else
    echo "Grade C"
fi
```

14.5 Loops

```
# for loop — iterate over a range
for i in {1..5}; do
    echo $i
done

# while loop — continue while condition is true
COUNT=1
while [ $COUNT -le 5 ]; do
    echo $COUNT
    COUNT=$((COUNT + 1))
done

# AWS example — check multiple servers
for SERVER in web1 web2 web3; do
    echo $SERVER
done
```

14.6 Functions

```
# Define a reusable function
greet() {
    echo "Hello $1"
}

# Call the function
greet Rahul
# Output: Hello Rahul
```

14.7 Cron Job Scheduling

Cron schedules commands to run automatically at defined times.

```
# Cron format:
# MIN HOUR DAY MONTH WEEKDAY command
# * * * * * /path/to/command

# Run every minute
* * * * * /home/ec2-user/script.sh

# Run daily at 2:00 AM
0 2 * * * /home/ec2-user/backup.sh

# Run every Sunday at midnight
0 0 * * 0 /home/ec2-user/report.sh

# Manage cron jobs
crontab -l # List all cron jobs
crontab -e # Edit cron jobs
```

14.8 Real AWS Scripts

Server Health Check

```
#!/bin/bash
echo "=== Server Health Report ==="
uptime
free -h
df -h
```

Disk Usage Alert

```
#!/bin/bash
USAGE=$(df -h | awk 'NR==2 {print $5}')
echo "Root disk usage: $USAGE"
```

⚠ WARNING: Always use full command paths in cron jobs (/usr/bin/df not df). Cron has a minimal PATH that may not find your commands.

Interview Questions

- Q What is a shebang?
- Q How do you pass arguments to a script?
- Q What is a cron job?
- Q What is the difference between a for and while loop?
- Q What is exit status \$?

Chapter Summary

In this chapter you learned:

- ✓ Bash scripts start with `#!/bin/bash` and require `chmod +x` to execute
- ✓ Variables store data; `read` accepts input; `$1`, `$2` receive arguments
- ✓ `if/elif/else` implements decision logic; `for/while` implement repetition
- ✓ Functions encapsulate reusable code blocks
- ✓ Cron schedules tasks using `MIN HOUR DAY MONTH WEEKDAY` format

CHAPTER 15

Linux Troubleshooting & Interview Preparation

Learning Objectives

After completing this chapter, you will be able to:

- Apply a structured 6-step troubleshooting methodology
- Diagnose and resolve common Linux and AWS EC2 problems
- Use the right diagnostic commands for CPU, memory, disk, and network issues
- Prepare answers for the top 50 Linux interview questions
- Conduct mock interview troubleshooting scenarios

15.1 Golden Troubleshooting Method

Step	Action
1. Identify	Clearly define the symptom and scope of the problem
2. Gather Info	Collect system data before making any changes
3. Analyse	Review logs, metrics, and configuration
4. Implement Fix	Apply the minimum change required
5. Verify	Confirm the problem is fully resolved
6. Document	Record the cause, fix, and prevention steps

15.2 Troubleshooting Cheatsheet by Category

Problem Area	Commands to Use
High CPU	uptime → top (press P) → ps aux --sort=-%cpu head
High Memory	free -h → top (press M) → ps aux --sort=-%mem head
Full Disk	df -h → du -sh /* → find / -type f -size +500M
Service Down	systemctl status → journalctl -u → systemctl restart
No Network	ip addr → ping 8.8.8.8 → ip route → ss -tulnp
DNS Failure	ping 8.8.8.8 → nslookup google.com → cat /etc/resolv.conf
Cannot SSH	EC2 status → systemctl status sshd → ss grep 22 → Security Group

Problem Area	Commands to Use
Website Down	systemctl status httpd → ss grep 80 → curl localhost → SG check
Permission Denied	ls -l → chmod → chown
EBS Not Visible	lsblk → dmesg grep nvme → fdisk → mkfs → mount

15.3 Top 50 Interview Questions

Linux Basics

- Q What is Linux?
- Q What is the Linux kernel?
- Q What is a shell?
- Q What is the difference between CLI and GUI?
- Q What is Bash?

Files & Directories

- Q Difference between cp and mv?
- Q What does mkdir -p do?
- Q What does ls -lah show?
- Q What is the root directory?
- Q How do you find all .log files?

Permissions

- Q Explain chmod 755.
- Q What does chmod +x do?
- Q What is the difference between chown and chgrp?
- Q What is SUID?
- Q What is the Sticky Bit and where is it used?

Users & Groups

- Q Difference between sudo and su?
- Q What is /etc/passwd?
- Q What is /etc/shadow?
- Q How do you add a user to a group?
- Q How do you delete a user AND their home directory?

Processes

- Q What is PID?
- Q Difference between kill and kill -9?
- Q What does nohup do?
- Q What is a daemon?
- Q How do you find the PID of a process by name?

Networking

- Q What is DNS?
- Q Difference between public and private IP?
- Q How do you check listening ports?
- Q What does curl -I do?
- Q Difference between nslookup and dig?

Storage

- Q Difference between df and du?
- Q What is EBS?
- Q What is a mount point?
- Q What is UUID and why use it in fstab?
- Q How do you troubleshoot 'disk full' error?

Services

- Q What is systemd?
- Q Difference between start and enable?
- Q What does journalctl -u do?
- Q What is PID 1?
- Q Difference between restart and reload?

Scripting

- Q What is Bash scripting?
- Q What is a shebang?
- Q Difference between for and while loop?
- Q What is a cron job?
- Q What is exit status \$??

Troubleshooting

- Q How do you troubleshoot a slow server?
- Q How do you troubleshoot a full disk?
- Q How do you troubleshoot SSH issues?
- Q How do you troubleshoot website inaccessibility?
- Q How do you troubleshoot DNS failures?

15.4 Mock Interview Scenarios

Q: Website is down. What do you check?

```
1. systemctl status httpd / nginx
2. ps -ef | grep httpd
3. ss -tulnp | grep 80
4. curl localhost
5. Check AWS Security Group — port 80 open?
6. journalctl -u httpd
```

Q: Server is very slow. How do you investigate?

```
1. uptime — check load averages
2. top — find CPU-hungry processes
3. free -h — check memory availability
4. df -h — check if disk is full
5. iostat -x — check disk I/O
6. journalctl -xe — look for errors
```

Q: Cannot SSH into EC2. What do you do?

```
1. AWS Console — verify instance is Running
2. systemctl status sshd (from console or another session)
3. ss -tulnp | grep 22 — is port 22 open?
4. Security Group — allow port 22 from your IP
5. Network ACL / Route Table — route to Internet Gateway
6. Correct key pair and correct username (ec2-user / ubuntu)
```

15.5 AWS Support Interview Tips

 **TIP:** Always mention checking logs (journalctl, tail -f) — interviewers value log awareness.

 **TIP:** Always describe VERIFICATION after your fix. Never stop at 'I restarted the service.'

 **TIP:** Know the difference between active (running now) and enabled (starts at boot).

 **WARNING:** Never say 'I would run rm -rf' without first verifying the path with pwd and ls.

Chapter Summary

In this chapter you learned:

- ✓ Always follow the 6-step troubleshooting method: Identify → Gather → Analyse → Fix → Verify → Document
- ✓ Match the right diagnostic commands to each problem category
- ✓ The top 50 interview questions cover all 15 handbook chapters
- ✓ Mock scenarios train you to think out loud — essential for technical interviews
- ✓ Verification and log awareness separate good candidates from great ones

Congratulations!

You have completed the Linux Fundamentals Handbook for AWS re/Start. You are now equipped with practical knowledge for:

- **AWS re/Start Programme**
- **AWS Cloud Support Associate**
- **Linux System Administrator**
- **Technical Support Engineer**
- **Cloud Operations Engineer**
- **DevOps Beginner Roles**

Master the commands. Practice daily. Think through problems — not just memorise answers.